

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is light green. They are positioned diagonally, with the blue one partially covering the green one.

Algoritma & Struktur Data

Week 5 - Sorting



Last Week Review

- Double Linked list
 - Initialization
 - Set head and tail to NULL
 - Creation
 - If NULL:
 - Set next and prev of new node to NULL
 - Set the head pointer to the new node
 - If not NULL
 - Set next and prev of new node accordingly (depending on the position of creation Front or back)
 - Change the head to current new node if we add it on the front
 - Change the tail to current new node if we add it on the back
 - Print
 - Set the current pointer to point to the head
 - While current pointer is not NULL
 - Print the current->value
 - Set the current to the current->next
 - Deletion
 - There will be three different case
 - Empty List (immediately return)
 - Only one node exist (delete that very node)
 - Delete in front or in the tail (adjust head and tail accordingly)
 - In the center (make use next->prev and prev->next notation)



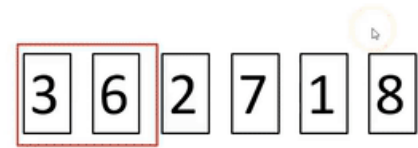
Outline

- Sorting
 - Bubble Sort
 - Concept
 - Implementation
 - Selection Sort
 - Concept
 - Implementation
 - Insertion Sort
 - Concept
 - Implementation
 - Radix Sort
 - Concept
 - Implementation

Bubble sort (concept)

- Imagine Bubbly water.
- Based on the size the bubble speed may vary.
- Imagine the size equal to the numerical value.
- The algorithm works by comparing current value with the next element of itself.
- If they find something (bigger/smaller), it will swap the data.
- The number of sorting iteration would be $N-1$;
- However the complexity would be N square. Meaning 4 data will have 3 iterations with 16 total of iteration.

Bubble Sort:



Implementation

- We start by making the first loop to retrieve the first value we wanted to compare.
- The second loop would be used to retrieve second value that we wanted to compare.
- Next we compare the first and second value and if the swap condition was met. Swap the value.
- Small question: guess the code. Is it going to be ASCENDING? Or DESCENDING?
- Optimization?

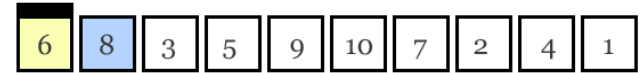
```
#include <stdio.h>
#include <stdlib.h>

int * bb_sort(int *arr,int start, int end){
    for(int i = start; i < end; i++){
        for(int j = start; j < end-1; j++){
            if(arr[j] > arr[j+1]){
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    return arr;
}

int main(void) {
    int *arr;
    arr = (int *) malloc(sizeof(int)*100);
    for(int i = 0; i < 100; i++){
        arr[i] = 500 + rand()%1000;
    }
    arr = bb_sort(arr,0,100);
    for(int i = 0; i < 100; i++){
        printf("%d ",arr[i]);
    }
    free(arr);
    return 0;
}
```

Selection sort (concept)

- Imagine you are playing poker and you wanted to sort your hand.
- You will check your card one by one
- If you see some value is bigger/smaller than your current card that you want to swap. You will swap it.
- See the simulation on the right guys.
- Selection sort in some way is an optimized version of Bubble sort
- Number of iterations is $N-1$ and the overall complexity is N square.



Yellow is smallest number found

Blue is current item

Green is sorted list

Implementation

- The overall code is similar to the bubble sort.
- The only difference lies on these points
 - We do not compare only on adjacent element.
 - We take i as a first element
 - We compare it with the rest of element with index larger than i .
 - This is symbolized by $start+i$.
- Alternative approach?

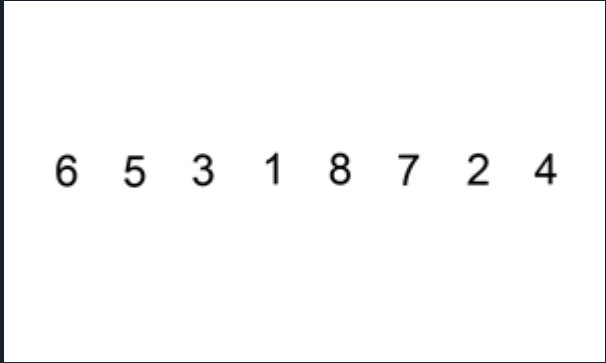
```
int * sel_sort(int *arr, int start, int end){
    for(int i = start; i < end; i++){
        for(int j = start+i; j < end; j++){
            if(arr[i] > arr[j]){
                int temp = arr[j];
                arr[j] = arr[i];
                arr[i] = temp;
            }
        }
    }
    return arr;
}

int main(void) {
    int *arr;
    arr = (int *) malloc(sizeof(int)*100);
    for(int i = 0; i < 100; i++){
        arr[i] = 500 + rand()%1000;
    }
    arr = sel_sort(arr, 0, 100);
    for(int i = 0; i < 100; i++){
        printf("%d ", arr[i]);
    }
    free(arr);
    return 0;
}
```



Insertion sort (concept)

- As the name suggests, basically we make an insertion in the element which is smaller or bigger (depending on the ASC or DESC sort) than a current inspected element
- The main step is:
 - For each index in array
 - Take the index key
 - Compare it with the predecessor of that value.
 - If it's smaller/bigger swap it.
 - If it's not then slide the value to make room for smaller/bigger value
 - Check the simulation
- Complexity: worst case N^2 while best case is N



6 5 3 1 8 7 2 4

Implementation

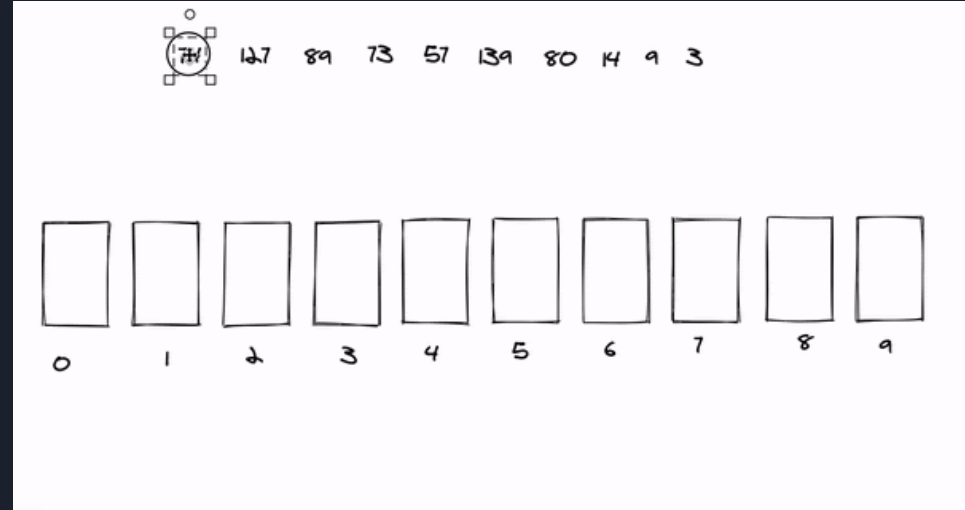
- Insertion code look far more simplified compared to two previous sorting algorithm.
- First we store our target value.
- Then we store the key as a j var.
- Then as long as there is a predecessor to check and the value of its predecessor is larger/smaller than key.
 - Slide the value j to j+1.
- The empty slot j+1 will be filled with the target value we store before

```
int * ins_sort(int *arr,int start, int end){
    start++;
    for(int i = start; i < end; i++){
        int temp = arr[i];
        int j = i-1;
        while(j >= start-1 && arr[j] > temp){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
    }
    return arr;
}

int main(void) {
    int *arr;
    arr = (int *) malloc(sizeof(int)*100);
    for(int i = 0; i < 100; i++){
        arr[i] = 500 + rand()%1000;
    }
    arr = ins_sort(arr,0,100);
    for(int i = 0; i < 100; i++){
        printf("%d ",arr[i]);
    }
    free(arr);
    return 0;
}
```

Radix sort (concept)

- Radix sort utilize a bucket and linked list concept. We represent this using an array of linked list.
- The number of iteration is defined by the longest element with most char or digit (depending on number or letter).
 - 123, 2, 50, 1222, 10.
 - These data will cause 4 iteration. Since element with largest digit is 1222.
- Number of bucket will be determined whether it's number or char.
 - Number means : 10 bucket with 0...9 representing digit
 - Char (all lowercase) : 26 bucket representing 26 alphabet.
 - Char (lower & Uppercase)?
- For each iteration it will store the element to the bucket based on the current digit checked.
 - If first iteration then we will check the digit for ones place value (satuan).
 - Second iteration will check the digit for tenth place value (puluhan).
- See the simulation on the left.





Radix Sort (Implementation)

- Basically we first prepare the array of linked list based on the variation of digit.
 - 26 for char
 - 52 for both upper and lower
 - 10 for integer
- Find the largest element
- Define the element with longest digit
- Make it as a number of iteration
- For each iteration we will get the LSD (least significant digit) top right.
 - We will put it on the bucket that match the current LSD.
 - Once all elements have been processed, put it back into the original array based on the order of the bucket.
 - Remember the insertion and deletion.
 - Repeat.
- Code (Quite Long):
 - <https://drive.google.com/file/d/1qfTuYov-XaGmbEwo3SHw32pVSH0UrmYo/view?usp=sharing>

Tugas Visualisasi Sorting

- Setiap kelompok diminta untuk membuat visualisasi sorting untuk setiap algoritma sorting dalam slide ini. Buatlah visualisasi untuk setiap perubahan yang terjadi dalam array yang di-sorting. Array yang akan di-sorting memiliki 10 indeks, yang sudah terisi secara acak sbb :
- Tugas dikerjakan dalam bentuk .ppt per kelompok.

20	95	85	70	30	10	5	2	90	40
----	----	----	----	----	----	---	---	----	----



THE END